

My To-Do List

Atharva · AI backend · from the Vibration AI Roadmap and the 85-ticket sheet (both dated 18 September 2026), checked against the code on 21 September 2026

What this is

Of the 85 tickets, **43 are mine**. Laxman has the rest. My half is the part that works out what the numbers mean; his is the part that shows them on screen and the plumbing that serves them.

Everything below is written as what the system will be able to do once the ticket is finished, rather than which file changes. The VIK number is kept so it matches the ticket sheet.

Four things in the documents are already out of date

These were checked against the actual code and database today, not taken from the documents. Three are good news; one is a problem to fix before I write any migration.

What the document says	What the code says today
Missing data files (VIK-002)	Already fixed. bearings.json, iso10816_3.json and fault_signatures.json are on disk and the three modules import without error. The ticket says every lookup fails on first call. It does not any more. Worth a quick re-check, then close it.
"No tests at all in the platform"	No longer true. The backend now has 4 test files and 58 passing tests . The discipline the roadmap asks for has started.
"Nine captures, the same synthetic file nine times"	No longer true. There are now 55 real captures from the live gateway on the actual pump. The thing the roadmap calls the critical path is no longer blocked.
Migration number 019 (VIK-006)	Number already taken. 019_raw_channel_summary_stats already exists. My migrations must start at 020 , and every later migration number in the ticket sheet shifts by one. Fix this before writing any migration or two people will create the same number.

The one that matters most

VIK-005 — teach the system what its own numbers mean. Right now a reading is stored exactly as it arrived, with no record of whether it is volts or g. Those differ by a factor of ten. Read one way the test pump measures 34 mm/s; read the other way, 344. The level where a machine is damaging itself is about 7 to 11.

Until this is done, every severity rating, every ISO grade and every comparison between two machines is wrong — and wrong in a way that looks completely normal on screen. Nothing else I build is trustworthy until this is.

Phase 0 — Clear the blockers

Nothing in Phase 1 starts until VIK-005 and VIK-009 are merged. These are not new features; they are things that are wrong now, and anything built on top inherits the error.

ID	What you are making it do	Size	Why it matters / what it needs first
VIK-002	Make the chatbot's reference data load reliably	M	Looks already done — the files are there and import fine. Verify with a test that calls one lookup in each of bearing, ISO and fault rules, then close.
VIK-003	Declare jinja2 so reports work on a fresh install	S	Still real. It is not in requirements.txt; it only works today because it came in by accident. Report rendering breaks on any clean setup.
VIK-004	Move the unit-conversion code into the backend	M	Copy it, do not rewrite it — it is already tested against published values. Keep its warning that broadband conversion is invalid. Needed by VIK-005 and VIK-007.
VIK-005	Make every reading carry its real unit	L	The most important ticket I have. Divide raw samples by the sensor's sensitivity and record the unit. Where sensitivity is missing, write "unconfirmed" and let later steps refuse — never guess. Needs VIK-004 and VIK-006.
VIK-006	Add two columns: what unit, and is it confirmed	S	Use migration 020, not 019. Existing rows become "unconfirmed" rather than assuming g.
VIK-007	Convert acceleration to velocity	M	Reuse the integration that already exists in the orbit code. A second one would drift from the first. Needs VIK-005. Without it there are no ISO grades.
VIK-008	Make a table to hold bearings	S	Migration 021 now. Manufacturer, designation, element count, the four fault orders.
VIK-009	Load 88,734 real bearings from the spreadsheet	M	Blocked: the spreadsheet is not in the repo. Ask Laxman for it. Today we know 2 bearings and estimate the rest with a warning that the ball count may be wrong by one.
VIK-010	Match each machine's bearings to the catalogue	M	Tolerate the suffixes plants write (6312-C3 means 6312). Where there is no match, fall back to the estimate and keep its lower confidence. Needs VIK-009.
VIK-014	Make the maths ten times faster	M	One 8-channel capture currently does about 2,560 spectrum calculations. Inverting two loops drops it to 256. This is what makes 50 features affordable later.
VIK-017	Build fake faults with known answers	M	Without this the diagnosis engine can only be demonstrated, never proven right. It is the scoring ground truth for everything from VIK-018 to VIK-053.

Phase 1 — Measure properly, and learn what normal is

I carry most of this phase. The end result: the system knows about fifty things about every capture, knows whether it can trust them, and holds a real picture of normal for each machine.

ID	What you are making it do	Size	Why it matters / what it needs first
VIK-018	Add 13 more time-domain measurements	L	Peak-to-peak, skewness, impulse and shape factors, burst count, shock index and others. Two already exist in the raw analysis code — lift them, do not write second copies.
VIK-019	Add 13 more frequency measurements	L	Harmonic count, sideband spacing, spectral spread, broadband noise and so on. Compute one spectrum per segment and reuse it; the current code recomputes the same FFT four times.
VIK-020	Add envelope measurements, including bearing bands	L	The single most important group for the demo. Reads energy at the exact bearing fault frequencies from the catalogue. Needs VIK-010 and VIK-019.
VIK-021	Register the new measurements in the database	M	Migration 022 now. Also: the defaults file and the migration hold the same numbers and are kept in step by hand — make one of them the only source.
VIK-022	Judge whether a capture can be trusted	L	Eight checks: clipping, saturation, noise floor, bias drift, missing data, unstable speed, loose sensor, odd DC offset. Returns High / Medium / Low / Invalid plus what failed. Every engine after this lowers its own confidence when trust is low.
VIK-024	Make a table for learned normal	S	Migration 023 now. Median, spread, percentiles, per sensor, channel, feature and operating mode.
VIK-025	Learn normal from history instead of one saved file	L	The foundation of the whole AI layer. Use median and MAD, not average and standard deviation, so one bad capture cannot move the baseline. Refuse below the minimum sample count rather than producing a weak baseline the system then trusts.
VIK-026	Baseline lifecycle: start, roll, freeze, reset	L	A reset must start a new version rather than editing the old one, so a finding can always be traced to the baseline that produced it.
VIK-027	Report how healthy the baseline itself is	M	Available or not, confidence, freshness, how many samples. This is the honest answer to today's situation, where the only baseline is a copy of the very file being compared against it.
VIK-035	Put the shared maths in one package both services use	L	Bearing maths, ISO tables, unit conversion, fault rules — 2,046 lines already tested. Stops the two services drifting into two different answers for the same bearing.

Phase 2 — Spot the unusual

Every reading gets a 0–100 "how unusual is this" score, judged against the right kind of normal. This is the first machine learning the platform has ever had.

ID	What you are making it do	Size	Why it matters / what it needs first
VIK-036	Stop the speed estimate locking onto the wrong peak	M	On channel 3 of the test pump it reported 50 Hz when the other seven channels agreed on 25. Check whether a candidate is double another before voting, then cross-check against the nameplate. A speed wrong by two puts every bearing frequency out by two.
VIK-037	Use the real tachometer reading when there is one	S	The column exists and is accepted on upload, but nothing ever reads it. A real reading should replace the estimate entirely.
VIK-038	Make tables for operating modes	S	Migration 024 now.
VIK-039	Work out which load the machine was running at	L	Simple banding first, then clustering. A capture with no clear mode is labelled unknown , never forced into the nearest one.
VIK-040	Keep a separate normal for each load	M	Without this, a machine that runs at two loads has a baseline that is the average of neither, and every score built on it is noise.
VIK-041	Make a table for the unusualness scores	S	Migration 025 now.
VIK-042	Score how unusual each reading is, 0–100	L	Robust statistics against the learned normal. A feature with no baseline scores unknown , never normal — that distinction must survive.
VIK-043	Add two machine-learning detectors	L	Isolation Forest and PCA residual, fitted per sensor , not one model across the fleet. Record which method contributed what so the score can be explained later.
VIK-044	Stop one odd reading becoming an alarm	M	Escalation needs repetition, a rising trend, steady speed and trustworthy data. Record each condition separately so the reason for escalating can be audited.
VIK-046	Make the four sensitivity profiles actually change behaviour	M	Conservative, Balanced, Early Warning, Expert. Switching between them must visibly change how many findings the same captures produce.

Phase 3 — Name the fault (this is the demo)

Six fault families only, done properly: bearing, unbalance, misalignment, looseness, gear mesh, cavitation. The rules are data, not code, so widening to all 26 later is a data change.

ID	What you are making it do	Size	Why it matters / what it needs first
VIK-050	Turn a spectrum into the peak list the rules expect	M	This one ticket joins the two halves of the system. Reuse the peak finder that already exists and is tuned. Do not write a second one.
VIK-051	Spot the symptoms in the signal	L	Harmonic series, sidebands, subharmonics, bearing band energy, impacting. Store each symptom with the evidence that triggered it , not just a flag — the whole value of this phase is that a finding can be defended.
VIK-052	Feed machine details to the fault rules automatically	M	Rated speed, pole count, vanes, blades, gear teeth, bearings, foundation — all already in the database. The report tool currently asks the operator to type in three of these.
VIK-053	Turn on the fault-ranking engine	L	The ranking, the evidence structure and the axial penalty are already written and tested. This is wiring, not authoring.
VIK-054	Grade severity from 0 to 5	M	Normal, Watch, Early Fault Suspected, Developing, Severe, Critical. A stage rises only under the persistence conditions — never on one reading.
VIK-055	Replace the fake health score with a real one	L	Today it is a fixed lookup: normal = 100, warning = 60, critical = 20. Replace with severity, unusualness, symptoms, trend, alarms, data quality, criticality and history.
VIK-056	Give an ISO grade (Zone A–D)	M	Where the machine group is ambiguous, return every applicable table and say what is missing rather than picking one. Needs VIK-007 and VIK-035.
VIK-057	Make the table that holds a finding	M	Migration 026 now. This is the AI card as a database table.
VIK-058	Make a finding something that evolves, not a new row each time	L	Re-loading the same capture must not create a second finding. This is an explicit acceptance criterion, and it is what makes "time since first detected" mean anything.
VIK-059	Map each fault to the plots that prove it	S	A table, not code. Every plot named already exists in the frontend.

Phase 4 — Ranking the work

ID	What you are making it do	Size	Why it matters / what it needs first
VIK-065	Rank which machines need attention first	L	Over unusualness, severity, criticality, persistence, trend, data quality, alarm history and impact. Record why a machine ranks where it does , not just the rank — a queue nobody can interrogate is a queue nobody trusts.

Phase 5 — Reports, remaining life, versions

ID	What you are making it do	Size	Why it matters / what it needs first
VIK-071	Estimate remaining useful life	L	Default to refusing. "Not available due to insufficient history" must be the normal answer, and saying so is itself an acceptance criterion. A confident wrong number here costs more trust than no number.
VIK-072	Stamp every finding with which versions produced it	M	Model, baseline, feature extraction, processing. Without it a finding from six months ago cannot be explained.
VIK-073	Stop the report tool asking for what the database knows	M	Power rating, foundation and drive arrangement are all one join away. The tool's own findings report calls this the cheapest item on the list by a wide margin .
VIK-074	Put the report builder behind an API	M	Keep the fact-ledger check — it blocks any report containing a number that does not trace to a recorded fact, and that is the reason the reports can be trusted.
VIK-075	Build the nine report types	L	Daily, weekly, monthly, critical alarm, fault evolution, bearing, gearbox, fleet, management. All nine share the existing machinery.

Phase 6 — Edge and fleet (after the demo)

ID	What you are making it do	Size	Why it matters / what it needs first
VIK-083	Write the edge pre-screening script	L	Write the conformance test first, then the script. The edge and the cloud must agree on the same samples.
VIK-084	Compare machines against their peers, not just their own past	L	Catches a machine that has been unhealthy since the day it was installed — something comparing against its own history can never find, because its bad behaviour is its normal.

The order I should actually work in

The ticket sheet gives dependencies, not priority. This is the order that unblocks the most, soonest:

#	Ticket	Why this order
1	VIK-003	Ten minutes. Add jinja2 and reports stop breaking on fresh installs.
2	VIK-002	Probably already done. Write the test, prove it, close it.
3	Renumber my migrations to 020+	Before writing any migration. 019 is taken. Tell Laxman so we do not collide.
4	VIK-004 → VIK-006 → VIK-005	The units chain. Everything downstream is wrong until this is done.
5	VIK-014	Ten times faster, and it makes Phase 1 affordable. Independent of everything else.
6	VIK-007	Velocity, which unlocks ISO grading.

#	Ticket	Why this order
7	VIK-017	The fake-fault harness. Nothing after this can be proven correct without it.
8	VIK-008 → VIK-009 → VIK-010	Bearings. Chase Laxman for the spreadsheet today — it is not in the repo.

Two rules that apply to every one of my tickets

1. No ticket merges without a test. 2. No ticket merges that adds a second way to do something the codebase already does. There are already two copies of the upload sequence and a 458-line job runner nothing calls.